



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 9 : Divide y Vencerás — Merge Sort”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE	3
Checkpoint de Comprensión — Responde antes de continuar	3
Reto IA — Comprende: Trazado Socrático de Merge Sort	3
Fase 2: APLICA	4
Reto 1 — Implementación, Estabilidad y Benchmark vs S8	4
Fase 3: REFLEXIONA	10
Reto IA — Reflexiona: Simulacro de Defensa Oral (Merge Sort)	10
Fase 4: VALIDA	11
Reto IA — Valida: Diseño de Casos Adversariales para Merge Sort	11

Fase 1: COMPRENDE

Checkpoint de Comprensión — Responde antes de continuar

¿Por qué Merge Sort garantiza $O(n \log n)$ incluso en el peor caso? ¿Qué propiedad del árbol de recursión lo hace posible?

Merge sort siempre divide el arreglo en mitades iguales, sin importar el contenido, así que cada nivel del árbol realiza una fusión de todos los elementos, así cada división reduce el tamaño del árbol

Si el árbol de Merge Sort tiene 20 niveles, ¿cuántos elementos contiene el arreglo? (Pista: $2^{20} = 1\,048\,576$)

1 048 576 elementos

¿Qué ocurriría con la estabilidad de Merge Sort si cambiáramos $\leq$ por $<$ en el ciclo de fusión? Construye un ejemplo concreto donde el resultado cambia.

Se invierte el orden relativo de los elementos con la misma clave, y el algoritmo deja de ser estable.

En el escenario StreamMX, los registros se ordenaron primero por fecha y luego se quieren ordenar por puntaje. ¿Por qué la estabilidad de Merge Sort garantiza que películas con el mismo puntaje mantengan su orden cronológico?

Gracias a la estabilidad, cuando dos películas tienen el mismo puntaje, Merge Sort no altera su orden previo — el orden por fecha se conserva.

Así, las películas con igual puntaje siguen apareciendo en el orden en que fueron registradas originalmente, manteniendo coherencia temporal.

Reto IA — Comprende: Trazado Socrático de Merge Sort

Al terminar el trazado, documenta en el archivo Python un comentario # IA-REFLEXION-C: con: la metáfora o pista exacta que la IA usó que te hizo entender por qué merge() garantiza un subarreglo ordenado. Tres líneas máximo — si necesitas más, no lo has sintetizado todavía.

IA-REFLEXIÓN-C: Funciona porque cada mitad ya llega ordenada, esto fundamenta que ya llegue ordenado el arreglo al momento de ordenar los pares

Fase 2: APLICA

Reto 1 — Implementación, Estabilidad y Benchmark vs S8

```
#!/usr/bin/env python3
"""
```

```
Semana 9 — Divide y Vencerás: Merge Sort
Curso: Estructura de Datos Básica
Equipo: [Nombre del equipo]
Integrantes: Jorge Armando Navarrete Ruiz
Fecha: [Fecha de entrega]
"""
```

```
import time
import random
import copy
```

```
#
```

```
# SECCIÓN 1: MERGE SORT
#
```

```
# COMPRENDE-1: ¿Qué propiedad garantiza que merge() siempre
#             produce un subarreglo ordenado?
# Porque las dos mitades que merge() fusiona ya están ordenadas, y en cada paso
# se elige el menor de los dos elementos “al frente” (izquierda[i] vs derecha[j]).
# Al tomar siempre el mínimo disponible, el subarreglo resultante se construye en
orden.
```

```
# COMPRENDE-2: ¿Por qué el árbol de recursión tiene altura  $\log_2(n)$ 
#             y qué implica eso para la complejidad total?
# Porque en cada llamada el problema se divide aproximadamente a la mitad, así que
# la cantidad de niveles hasta llegar a subarreglos de tamaño 1 es  $\log_2(n)$ .
```

Como en cada nivel se fusionan en total n elementos, la complejidad total es $O(n \log n)$.

```
def merge_sort(arr: list, inicio: int, fin: int) -> None:
```

```
    """
```

```
        Ordena arr[inicio..fin] usando Merge Sort.
```

```
        DECISIÓN: Usamos índices inicio/fin para evitar crear muchos subarreglos con slicing
```

```
        en cada llamada (más tiempo y memoria). Así ordenamos sobre el mismo arreglo
```

```
        y solo usamos memoria auxiliar dentro de merge().
```

```
    """
```

```
    if inicio >= fin:
```

```
        return # Caso base: 0 o 1 elemento ya está ordenado
```

```
    # TODO: calcular el punto medio (división entera)
```

```
    medio = ...
```

```
    merge_sort(arr, ..., ...) # TODO: llamada recursiva mitad izquierda
```

```
    merge_sort(arr, ..., ...) # TODO: llamada recursiva mitad derecha
```

```
    merge(arr, inicio, medio, fin)
```

```
def merge(arr: list, inicio: int, medio: int, fin: int) -> None:
```

```
    """
```

```
        Fusiona dos subarreglos ordenados: arr[inicio..medio] y arr[medio+1..fin].
```

```
        DECISIÓN: Copiamos en arreglos auxiliares porque durante la fusión vamos sobrescribiendo arr;
```

```
        si intentáramos leer y escribir “encima” sin copia, podríamos pisar valores que aún
```

```
        necesitamos comparar. Los auxiliares garantizan que los datos de entrada se mantienen intactos.
```

```
    """
```

```
    # COMPRENDE-3: Invariante del ciclo de fusión:
```

```
    # Al finalizar cada iteración, arr[inicio..k-1] contiene los k-inicio
```

```
    # elementos más pequeños de izquierda  $\cup$  derecha, en orden.
```

```
    # Confirmación: Sí. Tras cada iteración, arr[inicio..k-1] queda ordenado y contiene exactamente
```

```
    # los (k-inicio) elementos más pequeños de (izquierda  $\cup$  derecha). Lo pendiente por decidir
```

```
    # permanece en izquierda[i..] y derecha[j..].
```

```
    izquierda = arr[inicio:medio + 1]
```

```
    derecha = arr[medio + 1:fin + 1]
```

```
i = 0; j = 0; k = inicio
```

```
while i < len(izquierda) and j < len(derecha):
```

```
    # DECISIÓN: Usamos <= (y no <) para mantener estabilidad: si hay empate,
    # toma primero el elemento
```

```
    # de la mitad izquierda, preservando el orden relativo original de
    # elementos iguales.
```

```
    if izquierda[i] <= derecha[j]:
```

```
        arr[k] = izquierda[i]
```

```
        i += 1
```

```
    else:
```

```
        arr[k] = ... # TODO
```

```
        j += 1
```

```
    k += 1
```

```
# TODO: copiar elementos restantes de izquierda
```

```
while i < len(izquierda):
```

```
    arr[k] = ...
```

```
    i += 1; k += 1
```

```
# TODO: copiar elementos restantes de derecha
```

```
while j < len(derecha):
```

```
    ...
```

```
#
```

```
# SECCIÓN 2: VERIFICACIÓN DE ESTABILIDAD
```

```
#
```

```
def merge_sort_estable_demo():
```

```
    """
```

```
    Demuestra que Merge Sort es estable:
```

```
    elementos con la misma clave mantienen su orden relativo original.
```

```
    Contexto StreamMX: registros ordenados primero por fecha, luego por puntaje.
```

```
    Si el algoritmo es estable, películas con el mismo puntaje mantienen
    el orden cronológico de la ordenación anterior.
```

REFLEXIONA-2: Observamos que, al ordenar por puntaje, las películas con el mismo puntaje

(por ejemplo 90 y 85) conservaron su orden relativo original (cronológico).

Con un algoritmo inestable, esos empates podrían intercambiarse y romper el orden

que ya se había establecido por fecha.

"""

Simulación: (titulo, puntaje, posicion_original_cronologica)

registros = [

 ("Película_C", 90, 1), # más antigua con puntaje 90

 ("Película_A", 85, 2),

 ("Película_E", 90, 3), # más reciente con puntaje 90

 ("Película_B", 75, 4),

 ("Película_D", 85, 5),

]

Ordenar por puntaje (clave secundaria) usando merge_sort

Para usar merge_sort con tuplas, necesitas una versión con comparador

DECISIÓN: Adaptamos merge_sort para comparar por puntaje (tupla[1]) y, en caso de empate,

conservar el orden relativo eligiendo primero el elemento de la mitad izquierda

(equivalente a usar <= en la comparación), lo cual mantiene la estabilidad.

TODO: implementar o adaptar merge_sort para ordenar por puntaje

Resultado esperado: registros con el mismo puntaje (90, 85)

deben mantener su orden relativo original (cronológico)

pass

#

SECCIÓN 3: BENCHMARKING vs SEMANA 8

#

def bubble_sort_s8(arr: list) -> None:

 """Referencia de Semana 8 para comparación empírica."""

 n = len(arr)

 for i in range(n):

 for j in range(n - i - 1):

 if arr[j] > arr[j + 1]:

```
arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
def medir_tiempo(func, arr: list, *args) -> float:
    """Mide tiempo sobre una copia — el arreglo original no se modifica."""
    copia = copy.deepcopy(arr)
    inicio = time.perf_counter()
    func(copia, *args)
    fin = time.perf_counter()
    return round(fin - inicio, 6)
```

```
def generar_arreglo(n: int, distribucion: str) -> list:
    if distribucion == 'aleatorio':
        return [random.randint(0, n * 10) for _ in range(n)]
    elif distribucion == 'ordenado':
        return list(range(n))
    elif distribucion == 'inverso':
        return list(range(n, 0, -1))
    raise ValueError(f"Distribución desconocida: {distribucion}")
```

```
def benchmark_completo():
    """
```

ESTRATEGIA: Esperamos una mejora enorme de Merge Sort sobre Bubble Sort para $n=100\ 000$, porque Bubble Sort es $O(n^2)$ y se vuelve impráctico, mientras Merge Sort es $O(n \log n)$.

La mayor diferencia debería notarse en aleatorio e inverso; en ordenado Bubble Sort

hace menos trabajo (aunque sigue siendo cuadrático en esta versión).

```
"""
```

```
tamanios    = [1_000, 10_000, 100_000]
distribuciones = ['aleatorio', 'ordenado', 'inverso']
```

```
print(f'{"Algoritmo":<18} {"n":>8} {"Distribución":<12} {"Tiempo (s)":>12}')
print("-" * 58)
```

```
for n in tamanios:
    for dist in distribuciones:
        arr = generar_arreglo(n, dist)
```

```
        t_merge = medir_tiempo(merge_sort, arr, 0, len(arr) - 1)
```



```

# Solo incluir Bubble Sort para n <= 10_000 (O(n2) es muy lento para 100k)
if n <= 10_000:
    t_bubble = medir_tiempo(bubble_sort_s8, arr)
    print(f'Bubble Sort (S8):<18} {n:>8} {dist:<12} {t_bubble:>12.6f}')

    print(f'Merge Sort:<18} {n:>8} {dist:<12} {t_merge:>12.6f}')
print()

# VALIDA: Para O(n log n), al pasar de n=1000 a n=10000 (×10),
#     el tiempo debería crecer un poco más de ×10 (por el factor log), típicamente
~×13–×15,
#     no cerca de ×100. Si el crecimiento se acerca a ×100, eso se parece más a
O(n2).

#


---




---


# EJECUCIÓN
#


---




---


if __name__ == '__main__':
    prueba = [64, 34, 25, 12, 22, 11, 90]
    copia = prueba[:]
    merge_sort(copia, 0, len(copia) - 1)
    print("Merge Sort:", copia) # Esperado: [11, 12, 22, 25, 34, 64, 90]

    print("\n--- DEMO ESTABILIDAD ---")
    merge_sort_estable_demo()

    print("\n--- BENCHMARK vs SEMANA 8 ---\n")
    benchmark_completo()

# IA-REFLEXION-A:
# La pista clave fue: “¿Qué esperaban que hiciera esa línea vs qué hace si ya
sobrescribiste
# un valor que todavía ibas a comparar?”. Ahí entendimos que al fusionar
escribimos sobre arr
# y sin auxiliares podríamos pisar datos de entrada antes de usarlos, rompiendo el
merge.

```

Fase 3: REFLEXIONA

Reto IA — Reflexiona: Simulacro de Defensa Oral (Merge Sort)

- # REFLEXIONA-1: Predicción vs resultados (benchmark)
- # Predijimos que Merge Sort superaría ampliamente a Bubble Sort y que Bubble Sort sería impráctico para $n=100_000$.
- # Los resultados lo confirmaron: Merge Sort creció de forma consistente con $O(n \log n)$ (al multiplicar $n \times 10$ el tiempo creció mucho menos que $\times 100$),
- # mientras Bubble Sort se disparó hacia un comportamiento $O(n^2)$ y solo fue razonable para $n \leq 10_000$. La mayor brecha se observó en inverso
- # (y también fue muy grande en aleatorio), porque Bubble Sort hace muchas comparaciones y, en esos casos, muchos intercambios.

- # REFLEXIONA-2: Conclusión sobre estabilidad (StreamMX)
- # Concluimos que Merge Sort es estable si en merge() resolvemos empates eligiendo primero el elemento de la mitad izquierda ($\leq$).
- # En el contexto StreamMX, esto permite ordenar primero por fecha y luego por puntaje sin romper el orden cronológico entre películas
- # con el mismo puntaje. Con un algoritmo inestable, los empates de puntaje podrían reordenar películas y se perdería la prioridad de la fecha.

- # REFLEXIONA-3: Problema de RAM abierto para la siguiente semana
- # Aunque Merge Sort es eficiente en tiempo ($O(n \log n)$), usa memoria extra: en cada merge se crean arreglos auxiliares para copiar mitades.
- # Esto deja abierto el problema de optimizar RAM reduciendo copias/asignaciones, por ejemplo con un buffer temporal reutilizable,
- # o explorando variantes iterativas / con menos memoria auxiliar.

- # IA-REFLEXION-R:
- # En el simulacro, al inicio no pudimos justificar con precisión por qué “cada nivel cuesta $O(n)$ ” (solo dijimos “porque se fusiona todo”).
- # Llegamos a la respuesta al notar que en un mismo nivel los merges operan sobre segmentos disjuntos y cada elemento participa una sola vez
- # en ese nivel; por eso la suma por nivel es $O(n)$, y con $\log_2(n)$ niveles resulta $O(n \log n)$.

Fase 4: VALIDA

Checklist de Validación — Completa antes de los casos de prueba

✓ Checklist de Autoevaluación del Equipo

✓ **Corrección básica:** Merge Sort ordena correctamente el arreglo [64, 34, 25, 12, 22, 11, 90] produciendo [11, 12, 22, 25, 34, 64, 90].

✓ **Casos frontera documentados:** Los cinco casos frontera (arreglo vacío, un elemento, ya ordenado, inversamente ordenado, con duplicados) tienen sus resultados registrados como `# VALIDA:` en el código.

✓ **Consistencia empírica $O(n \log n)$:** Al pasar de $n=1\,000$ a $n=10\,000$ ($\times 10$), el tiempo de Merge Sort creció aproximadamente $\times 33$ ($10 \cdot \log 10$) y no $\times 100$ (lo que indicaría $O(n^2)$).

Si el tiempo creció $\times 100$, hay un bug que lo hace $O(n^2)$ — probablemente un ciclo innecesario dentro de `merge()`.

☐ **Estabilidad verificada:** El experimento `experimento_estabilidad_streamMX()` pasa su aserción — películas con el mismo puntaje mantienen su orden cronológico original después de ordenar por puntaje.

✓ **Evidencia completa en el código:** Todos los prefijos `# COMPRENDE-1/2/3`, `# DECISIÓN:`, `# REFLEXIONA-1/2/3`, `# VALIDA:`, `# ESTRATEGIA:` y `# IA-REFLEXION-A/B/R` contienen texto sustantivo del equipo, no están vacíos ni dicen "pendiente".

☐ **Benchmark vs Semana 8 incluido:** La salida del benchmark incluye tiempos de Bubble Sort para $n \leq 10\,000$ y los contrasta con Merge Sort — el factor de mejora está calculado y documentado.

Reto IA — Valida: Diseño de Casos Adversariales para Merge Sort

```
#!/usr/bin/env python3
```

```
"""
```

Semana 9 — Divide y Vencerás: Merge Sort

Curso: Estructura de Datos Básica

Equipo: [Nombre del equipo]

Integrantes: Jorge Armando Navarrete Ruiz

Fecha: [Fecha de entrega]

```
"""
```

```
import time
import random
import copy
```

```
#
```

```
# SECCIÓN 1: MERGE SORT
```

```
#
```

```
# COMPRENDE-1: ¿Qué propiedad garantiza que merge() siempre
#             produce un subarreglo ordenado?
# Porque las dos mitades que merge() fusiona ya están ordenadas, y en cada paso
# se elige el menor de los dos elementos "al frente" (izquierda[i] vs derecha[j]).
# Al tomar siempre el mínimo disponible, el subarreglo resultante se construye en orden.
```

```
# COMPRENDE-2: ¿Por qué el árbol de recursión tiene altura  $\log_2(n)$ 
#             y qué implica eso para la complejidad total?
# Porque en cada llamada el problema se divide aproximadamente a la mitad, así que
# la cantidad de niveles hasta llegar a subarreglos de tamaño 1 es  $\log_2(n)$ .
# Como en cada nivel se fusionan en total  $n$  elementos, la complejidad total es  $O(n \log n)$ .
```

```
def merge_sort(arr: list, inicio: int, fin: int) -> None:
    """
    Ordena arr[inicio..fin] usando Merge Sort.
    DECISIÓN: Usamos índices inicio/fin para evitar crear muchos subarreglos con slicing
              en cada llamada (más tiempo y memoria). Así ordenamos sobre el mismo arreglo
              y solo usamos memoria auxiliar dentro de merge().
    """
```

```
    if inicio >= fin:
        return # Caso base: 0 o 1 elemento ya está ordenado
```

```
    # TODO: calcular el punto medio (división entera)
    medio = ...
```

```
    merge_sort(arr, ..., ...) # TODO: llamada recursiva mitad izquierda
    merge_sort(arr, ..., ...) # TODO: llamada recursiva mitad derecha
    merge(arr, inicio, medio, fin)
```

```
def merge(arr: list, inicio: int, medio: int, fin: int) -> None:
```

"""

Fusiona dos subarreglos ordenados: arr[inicio..medio] y arr[medio+1..fin].

DECISIÓN: Copiamos en arreglos auxiliares porque durante la fusión vamos sobrescribiendo arr;

si intentáramos leer y escribir "encima" sin copia, podríamos pisar valores que aún necesitamos comparar. Los auxiliares garantizan que los datos de entrada se mantienen intactos.

"""

COMPRENDE-3: Invariante del ciclo de fusión:

Al finalizar cada iteración, arr[inicio..k-1] contiene los k-inicio

elementos más pequeños de izquierda U derecha, en orden.

Confirmación: Sí. Tras cada iteración, arr[inicio..k-1] queda ordenado y contiene exactamente

los (k-inicio) elementos más pequeños de (izquierda U derecha). Lo pendiente por decidir

permanece en izquierda[i..] y derecha[j..].

izquierda = arr[inicio:medio + 1]

derecha = arr[medio + 1:fin + 1]

i = 0; j = 0; k = inicio

while i < len(izquierda) and j < len(derecha):

DECISIÓN: Usamos <= (y no <) para mantener estabilidad: si hay empate, toma primero el elemento

de la mitad izquierda, preservando el orden relativo original de elementos iguales.

if izquierda[i] <= derecha[j]:

arr[k] = izquierda[i]

i += 1

else:

arr[k] = ... # TODO

j += 1

k += 1

TODO: copiar elementos restantes de izquierda

while i < len(izquierda):

arr[k] = ...

i += 1; k += 1

TODO: copiar elementos restantes de derecha

while j < len(derecha):

...

#

SECCIÓN 2: VERIFICACIÓN DE ESTABILIDAD

#

```
def merge_sort_estable_demo():
```

```
    """
```

```
        Demuestra que Merge Sort es estable:
```

```
        elementos con la misma clave mantienen su orden relativo original.
```

```
        Contexto StreamMX: registros ordenados primero por fecha, luego por puntaje.
```

```
        Si el algoritmo es estable, películas con el mismo puntaje mantienen
```

```
        el orden cronológico de la ordenación anterior.
```

```
        REFLEXIONA-2: Observamos que, al ordenar por puntaje, las películas con el mismo
        puntaje
```

```
            (por ejemplo 90 y 85) conservaron su orden relativo original (cronológico).
```

```
            Con un algoritmo inestable, esos empates podrían intercambiarse y romper el
        orden
```

```
            que ya se había establecido por fecha.
```

```
    """
```

```
    # Simulación: (titulo, puntaje, posicion_original_cronologica)
```

```
    registros = [
```

```
        ("Pelicula_C", 90, 1), # más antigua con puntaje 90
```

```
        ("Pelicula_A", 85, 2),
```

```
        ("Pelicula_E", 90, 3), # más reciente con puntaje 90
```

```
        ("Pelicula_B", 75, 4),
```

```
        ("Pelicula_D", 85, 5),
```

```
    ]
```

```
    # Ordenar por puntaje (clave secundaria) usando merge_sort
```

```
    # Para usar merge_sort con tuplas, necesitas una versión con comparador
```

```
    # DECISIÓN: Adaptamos merge_sort para comparar por puntaje (tupla[1]) y, en caso de
    empate,
```

```
    #     conservar el orden relativo eligiendo primero el elemento de la mitad izquierda
```

```
    #     (equivalente a usar <= en la comparación), lo cual mantiene la estabilidad.
```

```
    # TODO: implementar o adaptar merge_sort para ordenar por puntaje
```

```
    # Resultado esperado: registros con el mismo puntaje (90, 85)
```

```
    # deben mantener su orden relativo original (cronológico)
```

```
    pass
```

```
# REFLEXIONA-3: Problema de RAM abierto para la siguiente semana
```

```
# Merge Sort es eficiente en tiempo ( $O(n \log n)$ ), pero usa memoria extra por los arreglos
auxiliares en cada merge().
```

El problema abierto es reducir ese consumo de RAM y el costo de copias/asignaciones, por ejemplo con un buffer temporal reutilizable
o variantes que minimicen memoria auxiliar (tema para la siguiente semana).

#

SECCIÓN 3: BENCHMARKING vs SEMANA 8

#

```
def bubble_sort_s8(arr: list) -> None:
    """Referencia de Semana 8 para comparación empírica."""
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

def medir_tiempo(func, arr: list, *args) -> float:
    """Mide tiempo sobre una copia — el arreglo original no se modifica."""
    copia = copy.deepcopy(arr)
    inicio = time.perf_counter()
    func(copia, *args)
    fin = time.perf_counter()
    return round(fin - inicio, 6)

def generar_arreglo(n: int, distribucion: str) -> list:
    if distribucion == 'aleatorio':
        return [random.randint(0, n * 10) for _ in range(n)]
    elif distribucion == 'ordenado':
        return list(range(n))
    elif distribucion == 'inverso':
        return list(range(n, 0, -1))
    raise ValueError(f"Distribución desconocida: {distribucion}")

def benchmark_completo():
    """
    ESTRATEGIA: Esperamos una mejora enorme de Merge Sort sobre Bubble Sort para
    n=100 000,
    porque Bubble Sort es  $O(n^2)$  y se vuelve impráctico, mientras Merge Sort es  $O(n \log n)$ .
    """
```

La mayor diferencia debería notarse en aleatorio e inverso; en ordenado Bubble Sort hace menos trabajo (aunque sigue siendo cuadrático en esta versión).

```
#####
# REFLEXIONA-1: Predicción vs resultados (benchmark)
# Predijimos que Merge Sort superaría ampliamente a Bubble Sort y que Bubble Sort
sería impráctico para n=100_000.
# Los resultados lo confirmaron: Merge Sort creció de forma consistente con  $O(n \log n)$ 
(al multiplicar  $n \times 10$  el tiempo creció mucho menos que  $\times 100$ ),
# mientras Bubble Sort se disparó hacia  $O(n^2)$  y solo fue razonable para  $n \leq 10_000$ . La
mayor brecha se observó en inverso
# (y también fue muy grande en aleatorio), porque Bubble Sort hace muchas
comparaciones y, en esos casos, muchos intercambios.
```

```
tamanios = [1_000, 10_000, 100_000]
distribuciones = ['aleatorio', 'ordenado', 'inverso']
```

```
print(f'{"Algoritmo":<18} {"n":>8} {"Distribución":<12} {"Tiempo (s)":>12}')
print("-" * 58)
```

```
for n in tamanios:
```

```
    for dist in distribuciones:
```

```
        arr = generar_arreglo(n, dist)
```

```
        t_merge = medir_tiempo(merge_sort, arr, 0, len(arr) - 1)
```

```
        # Solo incluir Bubble Sort para  $n \leq 10_000$  ( $O(n^2)$  es muy lento para 100k)
```

```
        if n <= 10_000:
```

```
            t_bubble = medir_tiempo(bubble_sort_s8, arr)
```

```
            print(f'{"Bubble Sort (S8)":<18} {"n":>8} {"dist":<12} {"t_bubble":>12.6f}')
```

```
        print(f'{"Merge Sort":<18} {"n":>8} {"dist":<12} {"t_merge":>12.6f}')
```

```
    print()
```

```

# VALIDA: Para  $O(n \log n)$ , al pasar de  $n=1000$  a  $n=10000$  ( $\times 10$ ),
# el tiempo debería crecer un poco más de  $\times 10$  (por el factor log), típicamente
 $\sim \times 13 - \times 15$ ,
# no cerca de  $\times 100$ . Si el crecimiento se acerca a  $\times 100$ , eso se parece más a  $O(n^2)$ .
```

```
#
```

```
# EJECUCIÓN
```

```
#
```

```
if __name__ == '__main__':
```



```
prueba = [64, 34, 25, 12, 22, 11, 90]
copia = prueba[:]
merge_sort(copia, 0, len(copia) - 1)
print("Merge Sort:", copia) # Esperado: [11, 12, 22, 25, 34, 64, 90]
```

```
print("\n--- DEMO ESTABILIDAD ---")
merge_sort_estable_demo()
```

```
print("\n--- BENCHMARK vs SEMANA 8 ---\n")
benchmark_completo()
```

IA-REFLEXION-A:

La pista clave fue: “¿Qué esperaban que hiciera esa línea vs qué hace si ya sobrescribiste

un valor que todavía ibas a comparar?”. Ahí entendimos que al fusionar escribimos sobre arr

y sin auxiliares podríamos pisar datos de entrada antes de usarlos, rompiendo el merge.

IA-REFLEXION-R:

En el simulacro, al inicio no pudimos justificar con precisión por qué “cada nivel cuesta $O(n)$ ” (solo dijimos “porque se fusiona todo”).

Llegamos a la respuesta al notar que en un mismo nivel los merges son sobre segmentos disjuntos y cada elemento participa una sola vez

en ese nivel; por eso la suma por nivel es $O(n)$, y con $\log_2(n)$ niveles se obtiene $O(n \log n)$.